Final Project Structure (Latest & Correct)

```
rag_saas/
│
├── app.py                      # FastAPI backend (JWT + API endpoints)
├── config.py                   # Central configuration (models, DB, secrets)
├── ingest.py                   # Document ingestion + FAISS index builder
├── retriever.py                # FAISS retriever loader (with safe
deserialization)
├── generator.py                # Hugging Face LLM (Flan-T5)
├── requirements.txt            # Dependencies
│
├── faiss_index/                # Generated vector DB (auto-created)
│   ├── index.faiss
│   └── index.pkl
│
├── data/
│   └── docs/                  # Your knowledge base (.txt files)
│       ├── doc1.txt
│       ├── doc2.txt
│       └── ...
│
├── frontend/                   # Simple UI (optional but good for demo)
│   ├── index.html
│   ├── main.js
│   └── style.css
│
├── database/
│   └── users.db              # SQLite DB (auto-created)
│
└── utils/                      # Optional utilities (for scaling)
    ├── logger.py
    └── metrics.py
```

What Each File Does (Interview Explanation)

`app.py`

- Main backend (FastAPI)
- Handles:
    - `/signup`
    - `/login`
    - `/ask`
- Implements:

- JWT authentication
- User management
- API orchestration

`config.py`

Central configuration:

```
EMBEDDING_MODEL = "sentence-transformers/all-MiniLM-L6-v2"
MODEL_NAME = "google/flan-t5-large"
DATABASE_URL = "sqlite:///./database/users.db"
SECRET_KEY = "your_secret"
```

Keeps your system **modular and configurable**

`ingest.py`

- Loads documents
- Splits into chunks
- Converts to embeddings
- Stores in FAISS

Run once:

```
python ingest.py
```

`retriever.py`

- Loads FAISS index
- Handles similarity search

Important fix you applied:

```
FAISS.load_local(..., allow_dangerous_deserialization=True)
```

`generator.py`

- Runs Hugging Face model

- Generates answers using context

Uses:

- `transformers`
- `Flan-T5`

`faiss_index/`

- Stores vector database
- Auto-created after ingestion

`data/docs/`

- Your knowledge base
- Add `.txt`, `.pdf`, etc.

`frontend/`

- Simple UI to:
    - Login
    - Ask questions
- Optional but great for demo

`database/users.db`

- Stores users
- Created automatically

`requirements.txt`

```
fastapi
uvicorn
sqlalchemy
passlib[bcrypt]
python-jose
```

```
langchain==1.2.13
langchain-core
langchain-community
langchain-huggingface
transformers
torch
sentence-transformers
faiss-cpu
unstructured
```

How Everything Connects (Important for Interview)

Flow:

1. **User logs in → gets JWT**
2. User sends query → `/ask`
3. Backend:
    - Retrieves relevant docs (FAISS)
    - Passes context to LLM
4. LLM generates grounded response
5. API returns answer

How to Run (Explain Clearly if Asked)

```
# 1. Install dependencies
pip install -r requirements.txt

# 2. Add documents
data/docs/*.txt

# 3. Build vector DB
python ingest.py

# 4. Run backend
uvicorn app:app --reload
```

How to Impress the Interviewer

Say this confidently:

> “I structured the project in a modular way separating ingestion, retrieval, generation, and API layers. This makes it scalable, maintainable, and production-ready.”

Bonus (High Impact Statement)

> “This architecture allows me to easily swap components — for example replacing FAISS with Pinecone or Flan-T5 with GPT-4 — without changing the overall system design.”
